DeepL

Subscribe to DeepL Pro to translate larger documents.
Visit www.DeepL.com/pro for more information.

Desarrollo FullStack Prueba de

Ingeniería de software

Tecnología

Wompi

Última revisión: 9/oct/2025 por

Javier Alejandro Pérezjavier.perez@wompi.com , Laura Ramírez

laura.ramirez@wompi.co

La historia

Esta prueba tiene como objetivo evaluar diversos aspectos del desarrollo front-end y back-end, como la claridad del código, la definición de API/endpoints, la simplicidad del diseño, la cobertura de las pruebas o el uso correcto del protocolo HTTP, en un escenario realista, el desarrollo de la interfaz de usuario, la gestión de datos, la infraestructura como código, los principios de desarrollo de software y el uso de la IA como asistente de codificación.

En esta prueba debes crear una **aplicación para un producto pagado con Wompi**. Se trata de la incorporación, donde obtenemos los datos de pago del cliente para realizar la transacción de un producto específico, obtenemos los datos de entrega del cliente, mostramos el resultado del proceso de pago y actualizamos los datos de stock[1].

Proceso empresarial general

1. Debe crear una interfaz de usuario o página para mostrar el producto y las unidades disponibles en el stock de su tienda (con descripción y precio), que serán comprados por el cliente.

2. Muestre al cliente el botón de tipo de pago *«Pagar con tarjeta de crédito»*. Esto abrirá una ventana modal en la que se solicitará la información de la tarjeta de crédito.

3. El cliente completará los datos de la tarjeta de crédito, que deberán ser validados (se valorará positivamente la presencia de los logotipos de detección de tarjetas MasterCard y VISA), y la información de entrega. *(Recuerde que los datos de la tarjeta de crédito deben ser falsos, pero deben seguir la estructura de las tarjetas de crédito).*

4. A continuación, se mostrará el resumen del pago, que incluye (el importe del producto, la tarifa base añadida siempre y los gastos de envío) con el botón de pago en un componente de fondo [2].

5. Cuando el usuario haga clic en el botón de pago:
    1. Cree una transacción en PENDING en su API backend y obtenga un número de transacción.
    2. Llame a la API de Wompi para completar el pago.
    3. Una vez completado o fallido el pago:
        1. Actualice la transacción con el resultado de la transacción en su backend.
        2. Asigne el producto que se entregará al cliente
        3. Actualice el stock de su producto.

6. Por último, muestra el resultado de la transacción realizada y redirige a la página del producto con el stock actualizado.

Wompi

Referencias

1. https://soporte.wompi.co/hc/es-419/articles/360057781394--C%C3%B3mo-pagar-con-tarjetas-a-trav%C3%A9s-de-Wompi-
2. https://m2.material.io/components/backdrop

Lahistoria

Tus responsabilidades

✓ El diseño general de la API y la arquitectura de la información (esquema de la base de datos, estructura de carpetas, etc.).

✓ Decidir qué datos deben solicitarse/responderse en cada punto final. (*Añadir la colección Postman o la URL pública de Swagger en README.md*)

✓ Decidir qué validaciones debe realizar cada punto final, pensando en situaciones reales que podrían surgir.

✓ Manejar de forma segura los datos confidenciales.

✓ La API que crees debe tener existencias, transacciones, clientes y entregas. Deben realizar diferentes tipos de solicitudes.

✓ Debes crear una interfaz de usuario o una página para mostrar el producto y las unidades disponibles en el stock de tu tienda.

✓ Tu aplicación debe ser resistente; la aplicación debe recuperar el progreso realizado por el cliente en caso de actualización.

✓ La atención al detalle es fundamental.

✓ Esta aplicación sigue un proceso comercial de 5 pasos en pantalla.

1. Página del producto→ 2. Información de la tarjeta de crédito/entrega→ 3. Resumen→ 4. Estado final→ 5. Página del producto

Wompi

Referencias

1. *https://soporte.wompi.co/hc/es-419/articles/360057781394--C%C3%B3mo-pagar-con-tarjetas-a-trav%C3%A9s-de-Wompi-*
2. *https://m2.material.io/components/backdrop*

Desarrollo

1. Frontend: Debe implementar una aplicación de página única (SPA) creada en ReactJS o VueJS. NO se permiten otros marcos de trabajo

   .

   1. La aplicación debe estar orientada a dispositivos móviles y, como candidato, debes crear una interfaz que se adapte a múltiples tamaños de pantalla. Haz que tu aplicación sea responsiva, pero céntrate en el diseño para móviles. (Referencia de tamaño mínimo de pantalla: iPhone SE (2020) 1334 x 750 píxeles). La interacción con la interfaz de usuario funcionará correctamente y deberá ajustarse a los límites de la interfaz.
   2. Es obligatorio el uso de Redux o Vuex, siguiendo las alineaciones de la arquitectura Flux en la medida de lo posible. Además, almacena los datos de las transacciones de pago de forma segura en el *estado de la aplicación* o en *el almacenamiento local.*
   3. El diseño de la experiencia de usuario (UX) puede crearse según su propio criterio.
   4. Utiliza los marcos CSS que prefieras, pero te recomendamos que utilices las API flexbox o grid[1].

2. Backend: Debes implementar servicios API Backend creados en JavaScript/TypeScript o Ruby. Con Nest.js, Grappe o Sinatra. NO se permiten Ruby on Rails ni otros marcos.
   1. La lógica de negocio NO debe gestionarse dentro de la capa de enrutamiento/controlador, sino que puede estar en una capa separada. Prueba la arquitectura hexagonal y el patrón de puertos y adaptadores.
   2. Prueba a utilizar la programación orientada a ferrocarriles (ROP) para tus casos de uso.
   3. Puede utilizar cualquier tipo de base de datos. Recomendamos utilizar PostgreSQL o DynamoDB. (*Añada el diseño del modelo de datos en README.md*)
   4. Puedes utilizar cualquier tipo de ORM y biblioteca de serialización.
   5. La base de datos debe estar alimentada con productos ficticios. NO es necesario un punto final para crear nuevos productos.

3. Las pruebas unitarias son obligatorias tanto en el frontend como en el backend para la competición de las pruebas, con una cobertura superior al 80 %. (*Añade los resultados en README.md y créalos con Jest*).

4. Publique la aplicación utilizando cualquier proveedor de servicios en la nube. Recomendamos AWS[2] (*con el nivel gratuito de AWS, AWS Lambda, ECS, EKS, CloudFront, S3, RDS o DynamoDB*).

Wompi

Referencias

1. https://css-tricks.com/snippets/css/a-guide-to-flexbox/
2. https://engineering.teknasyon.com/single-page-application-deployment-on-aws-4b53d08b3ad3

Herramientas de prueba (UAT)

1. API de Wompi. Para empezar, debe leer:
    1. https://docs.wompi.co/docs/colombia/inicio-rapido/
    2. https://docs.wompi.co/docs/colombia/ambientes-y-llaves/

2. Utilice la siguiente información para usar las claves API o para acceder a una cuenta creada para esta prueba. *Nota importante: este usuario podría ser utilizado por cualquier persona que realice la misma prueba y su sesión de usuario podría cerrarse si se accede desde una ubicación diferente al mismo tiempo, intente integrarse con las claves API.*

    - Es importante que las pruebas se realicen siempre en modo Sandbox.
    - El modo Sandbox se encuentra en el menú principal→ Developers
    - Usuario Wompi (NO MODIFIQUE LAS CREDENCIALES NI AÑADA 2MFA)
        - URL: https://login.staging.wompi.dev/
        - Usuario: smltrs00
        - Contraseña: ChallengeWompi123*
    - URL de la API
        - UAT_URL=https://api.co.uat.wompi.dev/v1
        - UAT_SANDBOX_URL=https://api-sandbox.co.uat.wompi.dev/v1
    - Claves API Sandbox
        - pub_stagtest_g2u0HQd3ZMh05hsSgTS2lUV8t3s4mOt7
        - prv_stagtest_5i0ZGIGiFcDQifYsXxvsny7Y37tKqFWg
        - stagtest_events_2PDUmhMywUkvb1LvxYnayFbmofT7w39N
        - stagtest_integrity_nAIBuqayW70XpUqJS4qf4STYiISd89Fp

3. Asegúrate de utilizar el entorno Sandbox. Esto significa que NO es necesario realizar transacciones con dinero real en el contexto de esta prueba.

4. Intente utilizar AI CLI Assist para realizar sus integraciones.

Consideraciones

✅ Recomendamos crear ramas y solicitudes de extracción por función.
✅ Recomendamos el uso de IA si es posible.
⚠️ Debes enviar tu código a GitHub como repositorio público únicamente, y NO utilices palabras como «Wompi» (nombre de la empresa) en tu repositorio, ni compartas tu solución con otros desarrolladores o candidatos.

Referencias

1. *https://css-tricks.com/snippets/css/a-guide-to-flexbox/*
2. *https://engineering.teknasyon.com/single-page-application-deployment-on-aws-4b53d08b3ad3*

Wompi

Entregables

1. Finalización de la aplicación frontend y la API backend
2. Enlace al repositorio GitHub con README.md actualizado.
3. Enlace de la aplicación AWS implementada y conectada con su API backend.

Criterios de evaluación

1. [5 puntos] por README completado correctamente.
2. [5 puntos] Imágenes que se renderizan rápidamente y evitan los límites de UI/UX.
3. [20 puntos] Funcionalidad completa del proceso de incorporación para el pago con tarjeta de crédito.
4. [20 puntos] por el correcto funcionamiento de la API.
5. [30 puntos] por una cobertura superior al 80 % con pruebas unitarias de backend y frontend
6. [20 puntos] por la aplicación y la API implementadas a través de cualquier proveedor de servicios en la nube.

—

1. [5 *puntos extra*] por el uso de alineaciones OWASP, HTTPS y encabezados de seguridad.
2. [5 *puntos extra*] por aplicaciones totalmente responsivas y que funcionan en diferentes navegadores.
3. [10 *puntos extra*] Conocimientos de CSS.
4. [10 *puntos extra*] Código limpio.
5. [10 *puntos extra*] por estructurar tu aplicación utilizando la arquitectura hexagonal con el patrón de puertos y adaptadores.
6. [10 *puntos extra*] por seguir el ROP.

Consideraciones

✅ Para completar la prueba, necesitas al menos 100 puntos.
✅ Si obtienes los puntos mínimos, continuarás con la etapa de la entrevista.
⚠️ Si se detecta que el repositorio no tiene progreso ni commits, o que es similar a la solución de otro candidato, la prueba se anulará automáticamente por fraude.

Wompi

Referencias
1. *https://observatory.mozilla.org/*

¡Dalo todo, candidato!

#Efecto Wompi